



SpeechRefiner: Towards Perceptual Quality Refinement for Front-End Algorithms

Sirui Li¹, Shuai Wang^{2,1,3,*}, Zhijun Liu¹, Zhongjie Jiang⁴, Yannan Wang⁴, Haizhou Li^{1,3}

¹School of Data Science, The Chinese University of Hong Kong, Shenzhen, China

²School of Intelligence Science and Technology, Nanjing University, Suzhou, China

³Shenzhen Research Institute of Big Data, Shenzhen, China

⁴Tencent Ethereal Audio Lab, Tencent, Shenzhen, China

223040027@link.cuhk.edu.cn, shuaiwang@nju.edu.cn

Abstract

Speech pre-processing techniques such as denoising, de-reverberation, and separation, are commonly employed as front-ends for various downstream speech processing tasks. However, these methods can sometimes be inadequate, resulting in residual noise or the introduction of new artifacts. Such deficiencies are typically not captured by metrics like SI-SNR but are noticeable to human listeners. To address this, we introduce SpeechRefiner, a post-processing tool that utilizes Conditional Flow Matching (CFM) to improve the perceptual quality of speech. In this study, we benchmark SpeechRefiner against recent task-specific refinement methods and evaluate its performance within our internal processing pipeline, which integrates multiple front-end algorithms. Experiments show that SpeechRefiner exhibits strong generalization across diverse impairment sources, significantly enhancing speech perceptual quality. Audio demos can be found at <https://speechrefiner.github.io/SpeechRefiner/>.

Index Terms: Speech restoration, speech perceptual quality, conditional flow matching

1. Introduction

Speech pre-processing front-ends are critical for intelligent systems functioning in real-world acoustic environments, where target human voices are often corrupted by reverberation, noise, and overlapping sound sources. Typical tasks involve enhancement techniques to reduce non-speech interference, such as de-noising [1] and de-reverberation [2], and separation techniques for multi-speaker scenarios, including blind speech separation [3, 4] and target speaker extraction [5].

The evaluation of these front-end algorithms predominantly relies on objective metrics like SI-SNR (Scale-Invariant Signal-to-Noise Ratio) [6], which compares processed signals with clean references. While SI-SNR optimization has driven performance improvements [7], this metric doesn't perfectly align with human perception. A significant challenge is that front-end pre-processing can introduce artifacts, distortions, or spectral blurring, negatively impacting the auditory experience [8, 9]—a critical issue for applications like smart conferencing and hearing aids where perceptual fidelity is essential. This challenge also intersects with the era of large-scale data processing. Automated pipelines for cleaning in-the-wild audio data [10, 11] often discard substantial portions of raw datasets after quality filtering (e.g., using DNSMOS [12] or SIGMOS [13]). Integrating front-end enhancement into such pipelines could significantly improve retention of usable data, creating synergies between algorithmic advances and data-driven applications.

To address deficiencies in front-end processing, one approach is to employ speech restoration techniques to repair audio signals degraded by various impairments. Advanced methods such as VoiceFixer [14] and Miipher [15] enhance traditional front-end capabilities by tackling complex degradation scenarios using sophisticated neural architectures. These restoration techniques aim to holistically improve the perceptual quality of the original audio, rather than specifically addressing the challenges introduced by the aforementioned front-end processing. In addition to these universal restoration methods, task-specific solutions have been developed to counteract the challenges introduced by front-end processing. These include additional refining modules tailored for speech enhancement [16, 17], dereverberation [16], and separation [18, 19, 20]. While effective in their specific domains, these solutions are often closely integrated with particular front-end algorithms. For example, StoRM [16] and Fast-GeCo [20] necessitate joint training with front-end processors, and Diffiner [17] relies on noise estimates from speech enhancement front-ends as supplementary inputs. This reliance on co-trained components and front-end-specific features increases computational complexity and reduces adaptability, thereby limiting their generalizability and utility across varied scenarios.

To overcome these limitations, we introduce SpeechRefiner, a novel audio restoration model utilizing Conditional Flow Matching (CFM) [21]. Unlike aforementioned approaches that often necessitate front-end-specific inputs or co-training, SpeechRefiner serves as a versatile post-processing solution that enhances the perceptual quality of audio signals from various front-end algorithms. We validated SpeechRefiner on speech processed by various front-ends, including enhancement, dereverberation, separation, and TSE. We first compared it against task-specific refinement models, then tested it on an internal front-end pipeline, which integrates multiple front-end algorithms. The results demonstrate that SpeechRefiner significantly improves perceptual quality and exhibits strong generalization performance even on unseen front-end algorithms.

2. SpeechRefiner

In this work, we propose SpeechRefiner, a standalone refinement module designed to address degradation caused by various speech processing front-ends.

2.1. Model overview

SpeechRefiner consists of two main components, as shown in Figure 1. First, a Conformer-based model predicts mel-spectrograms from the distorted speech output of the front-end processing algorithm. Then, a neural vocoder synthesizes the mel-spectrograms to restore the speech signal.

*: Corresponding author.

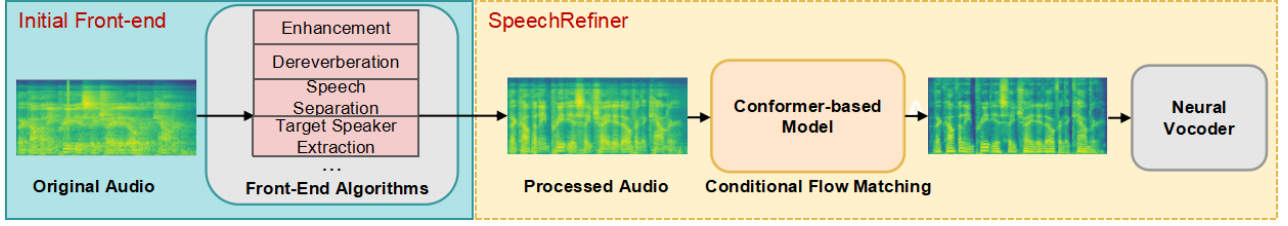


Figure 1: Overview of the SpeechRefiner Augmented Speech Restoration System.

SpeechRefiner acts as a post-processing module to address the distortions and artifacts introduced by front-end processing algorithms.

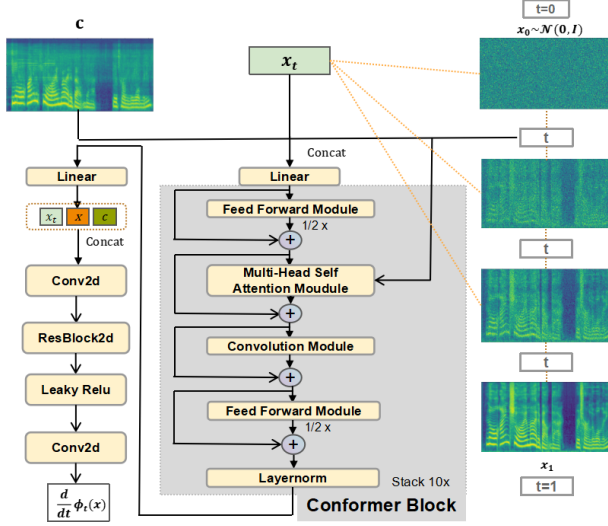


Figure 2: The architecture of the conformer-based model.

2.2. Conformer-based Model

2.2.1. Optimal-Transport Conditional Flow Matching

Modeling data distributions in high-dimensional spaces is non-trivial for neural networks, especially when the target true distribution $q(x)$ is unknown. One feasible solution is transforming a simple prior distribution p_0 , such as a Gaussian, into a more complex target distribution p_1 , which approximates $q(x)$. Flow matching [21] creates a continuous mapping, $\phi_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$, for $t \in [0, 1]$, which progressively shifts the prior distribution to the target by learning a time-dependent vector field $u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$. This transformation follows the dynamics defined by the ordinary differential equation (ODE):

$$\frac{d}{dt}\phi_t(x) = v_t(\phi_t(x)); \phi_0(x) = x. \quad (1)$$

However, directly calculating the vector field u_t and marginal density p_t is often computationally impractical, rendering flow matching difficult to apply. Conditional Flow Matching (CFM) [21] offers a solution by replacing the original vector field u_t with a conditional vector field $u_t(x|x_1)$, making the problem more manageable. The loss function for CFM is expressed as:

$$L_{CFM}(\theta) = \mathbb{E}_{t, q(x_1), p_t(x|x_1)} \|v_t(x; \theta) - u_t(x|x_1)\|^2. \quad (2)$$

Here, $p_t(x|x_1)$ represents the conditional distribution at time t , while $v_t(x; \theta)$ is a neural network that approximates $u_t(x|x_1)$. By leveraging conditional probabilities and fields, CFM avoids the difficulties of working with full marginal distributions. This

approach simplifies computations and ensures consistent gradient estimates, facilitating efficient training.

Building on the CFM framework, our model using Optimal-Transport Conditional Flow Matching (OT-CFM) [21], a variant of CFM that offers particularly straightforward gradients. The loss function for our model is defined as:

$$L(\theta) = \mathbb{E}_{t, q(x_1), p_0(x_0)} \|v_t(\phi_t(x)|c; \theta) - u_t(x_0|x_1)\|^2. \quad (3)$$

In this case, $\phi_t(x) = (1 - (1 - \sigma_{\min})t)x_0 + tx_1$ describes the flow from x_0 to x_1 , each x_1 paired with a sample $x_0 \sim N(0, I)$. Additionally, the learning target's gradient field $u_t(x_0|x_1) = x_1 - (1 - \sigma_{\min})x_0$. For our model, x_1 refers to clean speech, while c denotes distorted speech. The parameter $\sigma_{\min}$ is a small-valued hyperparameter set to zero in our experiments. The vector field $v_t(\phi_t(x)|c; \theta)$, which is the final output of our model, corresponds to the derivative $\frac{d}{dt}\phi_t(x)$ mentioned in the ordinary differential equation. This equation indicates how the randomly sampled Gaussian noise $x_0 \sim N(0, I)$ evolves at each time step t to match the target clean speech distribution. Essentially, this represents the slope of the flow at each time step.

2.2.2. The Conformer Architecture

The architecture of our conformer-based model is shown in Figure 2. The model takes x_t defined as the $\phi_t(x)$ introduced above, along with additional conditioning features $c \in \mathbb{R}^{N \times D_1 \times D_2}$ representing distorted speech and the current time step t . Here, N denotes the batch size, D_1 represents the number of frames, and D_2 is the number of Mel frequency bins. These inputs are concatenated and passed to Conformer blocks.

Each Conformer block [22] includes a feed-forward module, self-attention module, convolution module, and another feed-forward module. To capture long-range dependencies, we repeat this block 10 times. We replace the original relative position encoding with rotational position embedding (RoPE)[23], which is computationally efficient and generalizes better to longer sequences[24].

The Conformer output is transposed and concatenated with x_t and c , then processed by Conv2D and ResBlock2D layers. Each ResBlock2D contains two residual blocks with 2D convolutions and LeakyReLU activations. A final convolutional layer reduces the feature dimensions, producing the output $\frac{d}{dt}\phi_t(x) \in \mathbb{R}^{N \times D_1 \times D_2}$.

2.3. Neural Vocoder

After obtaining the refined mel-spectrogram, we use a pre-trained neural vocoder to reconstruct waveforms. Specifically, we adopt Vocos¹[25], which achieves state-of-the-art audio quality while significantly improving computational efficiency.

¹<https://github.com/gemelo-ai/vocos>

3. Experimental Setups

3.1. Front-end processing systems

We aim to test our system on speech processed by different front-end algorithms to evaluate its generalization performance. For each task, we randomly select 500 utterances from the corresponding test set to evaluate the proposed SpeechRefiner. The related front-end algorithms and the corresponding training data are summarized in Table 1, covering tasks such as speech enhancement, speech dereverberation, and speech separation, all of which have existing restoration modules proposed by previous studies. Moreover, to enable a more comprehensive comparative analysis, we also include two target speaker extraction systems (Audio based and Audio-Visual based).

Table 1: *Front-end algorithms and their corresponding datasets.*

Task	Front-end Algorithm	Refine Algorithm	Training Data
Enhancement	CDiffuSE [26]	Diffiner+ [17]	VB-DMD [27]
Dereverb	SGMSE+ [28]	StoRM [16]	WSJ0-REVERB [28]
Separation	Sepformer [29]	Fast-GeCo [20]	Libri2Mix [30]
Audio-TSE	Spex+ [31]	-	Libri2Mix [30]
AV-TSE	MuSE [32]	-	Vox2-Mix [32]

Specifically, both CDiffuSE² and SGMSE+³ are trained using the official implementations on the respective datasets, while Sepformer utilizes the checkpoint provided in the Fast-GeCo repository⁴.

3.2. Baseline refinement systems

For the baseline systems, we first adapt the well-known restoration toolkit VoiceFixer [14] to the setup of this study. We utilize the official implementation⁵. Notably, instead of the HiFi-GAN vocoder employed in the original paper, we use the Vocos vocoder across all systems to ensure consistency. Moreover, as discussed in Section 1, several refinement methods have been proposed for specific speech front-ends. We include them as additional front-end-specific baselines. The correspondence between algorithms, training data, and tasks is summarized in Table 1. Notably, we ensure that all baseline models, including VoiceFixer were trained and evaluated under the identical data conditions as SpeechRefiner for fair comparison.

3.3. Implementation Details

All audio files in our datasets are resampled to 24 kHz. The Short-Time Fourier Transform (STFT) is computed using a Hanning window with a window length of 1024 and a hop size of 256. The number of Mel frequency bins is set to 128.

For the Conformer model, we use the AdamW optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.95$, a weight decay of 0.01, and an initial learning rate of 1×10^{-4} . The model was trained for 500k steps, with a batch-size of 24.

Vocos is trained for up to 2500k steps with a batch size of 16. The AdamW optimizer is used with a learning rate of 2×10^{-4} , $\beta_1 = 0.9$ and $\beta_2 = 0.99$. A cosine annealing scheduler

is applied to adjust the learning rate. And the Vocos is trained on entire LibriTTS-R dataset [33].

During inference, following [34], a first-order Euler forward ODE solver is used for the flow matching model, and the number of inference steps is set to 64.

3.4. Evaluation metrics

We use SIGMOS[13] as our evaluation metric. The SIGMOS is released in the ICASSP 2024 SSI Challenge, which is adhered to the ITU-T P.804 standard. It serves as an objective metric approximating subjective human ratings and assesses speech quality across seven dimensions: Signal (SIG), Coloration (COL), Discontinuity (DISC), Loudness (LOUD), Noisiness (NOISE), Reverb (REVERB), and Overall Quality (OVRL). Trained on over 20,000 speech files, SIGMOS offers a standardized and automated approach for researchers to evaluate speech quality, identify degradation causes, and support studies in speech quality and communication networks.

4. Results and Analysis

4.1. Comparison with front-end-specific baselines

We first evaluate SpeechRefiner against state-of-the-art refiners designed for specific front-end tasks. Table 2 compares the performance across three core tasks: speech enhancement, dereverberation, and separation. Our SpeechRefiner outperforms or matches existing methods in most metrics. Notably, when we tested Diffiner+ (originally designed for speech enhancement) on the dereverberation task using a dereverberation dataset, its performance was significantly poor, with an overall score of 2.20, which highlights the limitation of these models being specifically designed for certain tasks. In contrast, SpeechRefiner achieves consistent improvements, highlighting its broader applicability. For example, on speech separation, SpeechRefiner achieves an OVRL score of 3.39, surpassing Fast-GeCo (3.03) by 11.9%. This underscores the advantage of SpeechRefiner’s task-agnostic design, which allows it to generalize across diverse front-end processing tasks without requiring task-specific modifications.

4.2. Training on the internal dataset

To further validate the effectiveness of the proposed SpeechRefiner, we utilize our internal front-end processing pipeline to simulate a scenario in which multiple algorithms are applied. In this experiment, we add noise⁶ and reverberation from the DNS-Challenge dataset⁷ to the LibriTTS-R dataset [33] (Training set merges the 100, 360 and 500 subset, and the test-clean set is used for testing). The distorted data is then processed using the internal front-end pipeline, which consists of denoising, acoustic echo cancellation (AEC), and dereverberation modules.

As shown in Table 3, SpeechRefiner consistently improves SIG and OVRL scores over unprocessed outputs. First, for the test set simulated using the same method as the training data, despite containing distortions introduced by two different front-end processing algorithms, a significant performance improvement is observed. The SIG and OVRL scores increase from 3.13 and 2.69 to 3.72 and 3.27, respectively.

Additionally, we evaluate SpeechRefiner, trained on the internal dataset, on several unseen test sets from the previous experiment. Compared to unprocessed results, our system

²<https://github.com/neillu23/CDiffuSE>

³<https://github.com/sp-uhh/sgmse>

⁴<https://github.com/WangHelin1997/Fast-GeCo>

⁵<https://github.com/haoheliu/voicefixer>

⁶Only non-speech noises were selected.

⁷<https://github.com/microsoft/DNS-Challenge>

Table 2: Performance of SpeechRefiner vs. Task-Specific Refiners Across Front-End Algorithms.

Front-end algorithm	System	COL	DISC	LOUD	NOISE	REVERB	SIG	OVRL
Enhancement: CDiffuSE	-	3.16	3.86	3.56	3.44	4.10	3.20	2.85
	VoiceFixer	3.49	4.26	3.94	3.89	4.45	3.57	3.23
	Diffiner+	3.10	4.10	3.62	4.14	4.44	3.35	3.02
	SpeechRefiner	3.44	4.34	3.74	4.09	4.55	3.66	3.32
Dereverberation: SGMSE+	-	3.67	3.91	4.00	3.63	4.39	3.68	3.18
	VoiceFixer	3.78	4.03	3.95	3.85	4.52	3.46	3.06
	Diffiner+	2.79	2.83	3.10	3.19	3.49	2.43	2.20
	StoRM	3.78	3.88	3.80	3.61	4.48	3.69	3.23
	SpeechRefiner	3.82	4.17	4.08	3.92	4.60	3.60	3.20
Separation: Sepformer	-	3.22	3.67	3.23	3.67	3.44	3.01	2.56
	VoiceFixer	3.17	3.92	3.87	3.57	4.29	3.27	2.86
	Fast-Geco	3.41	4.02	3.39	3.98	3.81	3.61	3.03
	SpeechRefiner	3.89	4.37	4.15	3.97	4.56	3.79	3.39

Table 3: SpeechRefiner Training on the Internal Dataset and Evaluation Across Front-End Algorithms

Front-end algorithm	Refinement	SIG	OVRL
Enhancement & Dereverb: Internal	×	3.13	2.69
	✓	3.72	3.27
Enhancement: CDiffuSE	×	3.20	2.85
	✓	3.53	3.13
Dereverberation: SGMSE+	×	3.68	3.18
	✓	4.06	3.73
Separation: Sepformer	×	3.01	2.56
	✓	3.45	3.00
Audio-TSE: Spex+	×	3.04	2.64
	✓	3.75	3.35
AV-TSE: MuSE	×	2.71	2.24
	✓	3.64	3.19

consistently demonstrates significant improvements. In some cases, its performance even surpasses that of systems trained on matched training data, highlighting the strong generalization capability of our approach.

Furthermore, we evaluate the AV-TSE and Audio-TSE and treat them as unseen front-end processing algorithms, including Audio-TSE with Spex+ and AV-TSE with MuSE, where it achieved remarkable performance.

4.3. Visualization of the refinement effectiveness

Finally, to better illustrate the effectiveness of the proposed SpeechRefiner, we present the audio spectrograms before and after restoration in Figure 3. By comparing the spectrograms dereverberated by SGMSE+ and extracted by the AV-TSE system with those restored by SpeechRefiner, we observe a significant improvement in clarity. SpeechRefiner effectively eliminates residual reverberation in SGMSE+ outputs and suppresses overlapping speech artifacts in AV-TSE results, confirming its ability to enhance perceptual clarity across diverse scenarios.

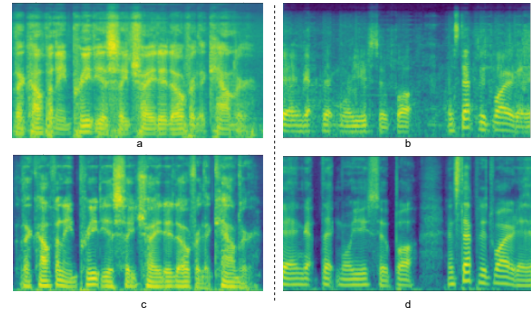


Figure 3: Spectrogram Visualization of SpeechRefiner a) and b) show the outputs of the SGMSE+ and AV-TSE systems, respectively; c) and d) depict the corresponding restored versions produced by SpeechRefiner.

5. Conclusion

In this paper, we proposed SpeechRefiner, a post-processing module aimed at mitigating impairments and artifacts introduced by various front-end processing algorithms, which significantly affect perceptual quality. Trained on data derived from a single impairment source, SpeechRefiner demonstrated remarkable generalization capabilities across a variety of unseen front-end algorithms. This underscores the model’s robustness despite limited training diversity. Its effectiveness highlights its potential as a complementary tool for improving perceptual quality.

Despite its promising performance, our experiments revealed two main limitations. First, the model struggles with restoring speech containing multiple speakers, often merging them into a single voice. Second, when processing severely distorted speech with limited intelligibility, SpeechRefiner tends to produce clean yet semantically meaningless audio signals.

To address these challenges in future work, we plan to (i) enhance the model’s ability to differentiate and restore multi-speaker inputs, (ii) refine its handling of severely degraded speech to preserve semantic accuracy, and (iii) increase the data diversity in the data simulation pipeline to create a more generalizable version of SpeechRefiner for open-sourcing.

6. Acknowledgements

This work was supported by National Natural Science Foundation of China, Grant No. 62401377, Shenzhen Science and Technology Program (Shenzhen Key Laboratory, Grant No. ZDSYS20230626091302006), Shenzhen Science and Technology Research Fund (Fundamental Research Key Project, Grant No. JCYJ20220818103001002), Program for Guangdong Introducing Innovative and Entrepreneurial Teams, Grant No. 2023ZT10X044.

7. References

- [1] A. Azarang and N. Kehtarnavaz, "A review of multi-objective deep learning speech denoising methods," *Speech Communication*, vol. 122, pp. 1–10, 2020.
- [2] P. A. Naylor and N. D. Gaubitch, *Speech dereverberation*. Springer Science & Business Media, 2010.
- [3] D. Wang and J. Chen, "Supervised speech separation based on deep learning: An overview," *IEEE/ACM transactions on audio, speech, and language processing*, vol. 26, no. 10, pp. 1702–1726, 2018.
- [4] Y. Luo and N. Mesgarani, "Conv-tasnet: Surpassing ideal time-frequency magnitude masking for speech separation," *IEEE/ACM transactions on audio, speech, and language processing*, vol. 27, no. 8, pp. 1256–1266, 2019.
- [5] K. Zmolikova, M. Delcroix, T. Ochiai, K. Kinoshita, J. Černocký, and D. Yu, "Neural target speech extraction: An overview," *IEEE Signal Processing Magazine*, vol. 40, no. 3, pp. 8–29, 2023.
- [6] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "Sdr-half-baked or well done?" in *ICASSP 2019*. IEEE, 2019, pp. 626–630.
- [7] Y. Luo and N. Mesgarani, "Tasnet: time-domain audio separation network for real-time, single-channel speech separation," in *ICASSP 2018*. IEEE, 2018, pp. 696–700.
- [8] M. Liu, Z. Chen, X. Yan, Y. Lv, X. Xia, C. Huang, Y. Xiao, and L. Xie, "Rad-net: A repairing and denoising network for speech signal improvement," *arXiv preprint arXiv:2401.04389*, 2024.
- [9] —, "Rad-net 2: A causal two-stage repairing and denoising speech enhancement network with knowledge distillation and complex axial self-attention," in *Proc. Interspeech 2024*, 2024, pp. 1700–1704.
- [10] J. Yu, H. Chen, Y. Bian, X. Li, Y. Luo, J. Tian, M. Liu, J. Jiang, and S. Wang, "Autoprep: An automatic preprocessing framework for in-the-wild speech data," in *ICASSP 2024*. IEEE, 2024, pp. 1136–1140.
- [11] L. Ma, D. Guo, K. Song, Y. Jiang, S. Wang, L. Xue, W. Xu, H. Zhao, B. Zhang, and L. Xie, "Wenetspeech4tts: A 12,800-hour mandarin tts corpus for large speech generation model benchmark," in *Proc. Interspeech 2024*, 2024, pp. 1840–1844.
- [12] C. K. Reddy, V. Gopal, and R. Cutler, "Dnsmos: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors," in *ICASSP 2021*. IEEE, 2021, pp. 6493–6497.
- [13] N.-C. Ristea, A. Saabas, R. Cutler, B. Naderi, S. Braun, and S. Branets, "Icassp 2024 speech signal improvement challenge," in *2024 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*. IEEE, 2024, pp. 15–16.
- [14] H. Liu, X. Liu, Q. Kong, Q. Tian, Y. Zhao, D. Wang, C. Huang, and Y. Wang, "Voicefixer: A unified framework for high-fidelity speech restoration," in *Proc. Interspeech 2022*, 2022, pp. 4232–4236.
- [15] Y. Koizumi, H. Zen, S. Karita, Y. Ding, K. Yatabe, N. Morioka, Y. Zhang, W. Han, A. Bapna, and M. Bacchiani, "Miipher: A robust speech restoration model integrating self-supervised speech and text representations," in *2023 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*. IEEE, 2023, pp. 1–5.
- [16] J.-M. Lemerrier, J. Richter, S. Welker, and T. Gerkmann, "Storm: A diffusion-based stochastic regeneration model for speech enhancement and dereverberation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.
- [17] R. Sawata, N. Murata, Y. Takida, T. Uesaka, T. Shibuya, S. Takahashi, and Y. Mitsufuji, "Diffiner: A versatile diffusion-based generative refiner for speech enhancement," in *Proc. Interspeech 2023*, 2023, pp. 3824–3828.
- [18] M. Hirano, K. Shimada, Y. Koyama, S. Takahashi, and Y. Mitsufuji, "Diffusion-based signal refiner for speech separation," *arXiv preprint arXiv:2305.05857*, 2023.
- [19] S. Lutati, E. Nachmani, and L. Wolf, "Separate and diffuse: Using a pretrained diffusion model for better source separation," in *12th International Conference on Learning Representations, ICLR 2024*, 2024.
- [20] H. Wang, J. Villalba, L. Moro-Velazquez, J. Hai, T. Thebaud, and N. Dehak, "Noise-robust speech separation with fast generative correction," in *Proc. Interspeech 2024*, 2024, pp. 2165–2169.
- [21] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *11th International Conference on Learning Representations, ICLR 2023*, 2023.
- [22] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," in *Proc. Interspeech 2020*, 2020, pp. 5036–5040.
- [23] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *Neuro-computing*, vol. 568, p. 127063, 2024.
- [24] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in *ICLR*, 2022.
- [25] H. Siuzdak, "Vocos: Closing the gap between time-domain and fourier-based neural vocoders for high-quality audio synthesis," *arXiv preprint arXiv:2306.00814*, 2023.
- [26] Y.-J. Lu, Z.-Q. Wang, S. Watanabe, A. Richard, C. Yu, and Y. Tsao, "Conditional diffusion probabilistic model for speech enhancement," in *ICASSP 2022*. IEEE, 2022, pp. 7402–7406.
- [27] C. Valentini-Botinhao, X. Wang, S. Takaki, and J. Yamagishi, "Investigating rnn-based speech enhancement methods for noise-robust text-to-speech," in *SSW*, 2016, pp. 146–152.
- [28] J. Richter, S. Welker, J.-M. Lemerrier, B. Lay, and T. Gerkmann, "Speech enhancement and dereverberation with diffusion-based generative models," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 2351–2364, 2023.
- [29] C. Subakan, M. Ravanelli, S. Cornell, M. Bronzi, and J. Zhong, "Attention is all you need in speech separation," in *ICASSP 2021*. IEEE, 2021, pp. 21–25.
- [30] J. Cosentino, M. Pariente, S. Cornell, A. Deleforge, and E. Vincent, "Librimix: An open-source dataset for generalizable speech separation," *arXiv preprint arXiv:2005.11262*, 2020.
- [31] M. Ge, H. Li, C. Xu, L. Wang, E. S. Chng, and J. Dang, "Spex+: A complete time domain speaker extraction network," *Interspeech 2020*, 2020.
- [32] Z. Pan, R. Tao, C. Xu, and H. Li, "Muse: Multi-modal target speaker extraction with visual cues," in *ICASSP 2021*. IEEE, 2021, pp. 6678–6682.
- [33] Y. Koizumi, H. Zen, S. Karita, Y. Ding, K. Yatabe, N. Morioka, M. Bacchiani, Y. Zhang, W. Han, and A. Bapna, "Libritts-r: A restored multi-speaker text-to-speech corpus," *arXiv preprint arXiv:2305.18802*, 2023.
- [34] V. Popov, I. Vovk, V. Gogoryan, T. Sadekova, and M. Kudinov, "Grad-tts: A diffusion probabilistic model for text-to-speech," in *International Conference on Machine Learning*. PMLR, 2021, pp. 8599–8608.